

How your theme system actually works

Three classes — `AppTheme` , `AppThemeScope` and `ThemeScopeWidget` — split one job into three. This walks through what each one does, how a value travels from startup to a widget, and what happens when the user flips dark mode.

`app_template` `lib/theme/` 3 classes · 2 flows · 6 findings

01 – Orientation

The one-sentence version

Each class answers a different question. Once you can name the question, the split stops feeling arbitrary.

PLAIN DATA CLASS

AppTheme

What the theme is. A bundle holding the four theme extensions — colors, typography, input theme, button theme — for one mode. No widget, no context, no state.

The paint set. Just the pigments in a box.

STATEFULWIDGET

ThemeScopeWidget

Which theme, and remembering it. Owns the current `ThemeMode` , saves it to disk, watches the OS brightness, and builds the matching `AppTheme` .

The decider. Picks a paint set and remembers your choice.

INHERITEDWIDGET

AppThemeScope

Getting it to widgets. Carries the chosen `AppTheme` down the tree so any descendant can read it from `context` without it being passed through constructors.

The delivery pipe. Runs paint to every room.

THE SHAPE OF IT

Data · Control · Distribution. That's the pattern. It's the same three-way split you'd find behind most state management, just written out by hand instead of hidden inside a package.

02 – The picture

Where each class sits in the tree

Read top to bottom. The important detail: `ThemeScopeWidget` lives *above* `MaterialApp`, not inside it — it is the widget `runApp()` receives.

runApp(themeScope)	Entry point <code>main.dart:13</code>
└─ ThemeScopeWidget (Stateful)	Owns the mode · reads & writes <code>SharedPreferences</code> <code>theme_mode_handler.dart</code>
└─ AppThemeScope (Inherited)	Carries the resolved <code>AppTheme</code> downward <code>app_theme_scope.dart</code>
└─ AppTemplate	Reads the scope, configures the app <code>app_template.dart:14</code>
└─ MaterialApp.router	Registers the four extensions into <code>ThemeData</code> <code>app_template.dart:42</code>
└─ your screens	<code>context.colors</code> · <code>context.typography</code> <code>theme_ext.dart:19</code>

That last hop is worth noticing. Your screens don't read `AppThemeScope` at all — they read `Theme.of(context).extension<AppColors>()`, wrapped in the `context.colors` getter. The scope's job finishes at `AppTemplate`.

What happens at startup

A real sequence, in order. Each step hands something to the next.

01 Load the saved preference — before the first frame

`await ThemeScopeWidget.initialize(const AppTemplate())` awaits `SharedPreferences.getInstance()` and returns a configured widget. Because this is awaited in `main()`, the saved mode is known before anything paints — that's what prevents a flash of the wrong theme on launch.

```
final themeScope = await ThemeScopeWidget.initialize(const AppTemplate());
await init();
runApp(themeScope);
```

02 Read the stored mode

`didChangeDependencies()` pulls the saved integer from preferences and turns it back into an enum. The default of `0` maps to `ThemeMode.system`, since Dart declares the enum as `{system, light, dark}`.

```
final themeModeIndex = widget.preferences.getInt(_kThemeMode) ?? 0;
_themeMode = ThemeMode.values[themeModeIndex];
```

03 Resolve "system" into a real brightness

`build()` asks the platform what it's currently set to, then picks a concrete `AppTheme`. This is the step that turns an *intent* (`system`) into an *answer* (light or dark).

```
final brightness = MediaQuery.platformBrightnessOf(context);

final appTheme = switch (_themeMode!) {
  ThemeMode.light => AppTheme.light(),
  ThemeMode.dark  => AppTheme.dark(),
  ThemeMode.system => brightness == Brightness.dark
    ? AppTheme.dark()
    : AppTheme.light(),
};
```

04 Publish it to the tree

The chosen bundle is wrapped in the `InheritedWidget`, which makes it reachable from any descendant's `context`.

```
return AppThemeScope(themeMode: _themeMode!, theme: appTheme, child: widget.child);
```

05 Hand the extensions to Material

`AppTemplate` reads the scope and registers the four extensions on `ThemeData`. From here on, Flutter's own theme machinery does the distributing.

```
final theme = AppThemeScope.of(context);
// ...
theme: ThemeData(
  scaffoldBackgroundColor: theme.theme.colors.surface,
  extensions: [theme.theme.colors, theme.theme.typography,
               theme.theme.inputTheme, theme.theme.buttonTheme],
),
```

06 A widget asks for a color

The extension on `BuildContext` is the last link — it's a thin convenience over `Theme.of(context)`, which is why any widget under `MaterialApp` can use it with no wiring.

```
AppColors get colors => theme.extension<AppColors>!;

// at the call site:
Container(color: context.colors.surface)
```

04 – Flow B

What happens when the user switches theme

One method call, then the framework does the rest. This is the payoff for the three-way split.

```
ThemeScopeWidget.of(context)!.changeTo(ThemeMode.dark);
```

01 Find the controller

`of()` uses `findRootAncestorStateOfType` to reach the state object. Note it deliberately does *not* subscribe the caller to rebuilds — you only want to call a method on it, not listen to it.

02 Persist first, then update

The index is written to disk *before* `setState`. If the write throws, the UI never claims a change that wasn't saved. There's also an early return when the mode is unchanged, so re-selecting the current theme costs nothing.

```
if (_themeMode == themeMode) return;

final index = ThemeMode.values.indexOf(themeMode);
await widget.preferences.setInt(_kThemeMode, index);

setState(() { _themeMode = themeMode; });
```

03 Rebuild cascades down

`setState` rebuilds `ThemeScopeWidgetState`, which builds a fresh `AppTheme`, which goes into a new `AppThemeScope`. Because `updateShouldNotify` returns true, everything depending on the scope rebuilds — `AppTemplate` included.

04 New ThemeData, whole tree repaints

`AppTemplate` builds new `ThemeData` with the new extensions. Every `context.colors` call in the app now returns the dark values — and because each extension implements `lerp`, the change animates rather than snapping.

05 – Reference

Class by class

AppTheme — the data

A `final class`, not a widget. It holds four extensions and nothing else. The two factories build a full set from a colour palette, so `AppColors` is the single source everything else derives from.

`lib/theme/app_theme.dart`

```
factory AppTheme.light() {
  final colors = AppColors.light();
  return AppTheme(
    colors: colors,
    typography: AppTypography.light(colors),
    inputTheme: AppInputTheme.light(colors),
    buttonTheme: AppButtonTheme.light(colors),
  );
}
```

Because it has no behaviour, it's trivially testable and can be constructed anywhere — no `BuildContext` required.

ThemeScopeWidget — the controller

The only stateful piece, and the only one that touches storage. It has three responsibilities: **remember** (`SharedPreferences`), **observe** (platform brightness), and **resolve** (`ThemeMode` → concrete `AppTheme`).

MEMBER	PURPOSE
<code>initialize()</code>	Async setup — awaits <code>SharedPreferences</code> so the saved mode is ready before the first frame.
<code>of(context)</code>	Reaches the state object so callers can invoke <code>changeTo</code> . Does not create a rebuild dependency.
<code>changeTo(mode)</code>	Persists the new mode, then <code>setState</code> . Early-returns when unchanged.
<code>didChangeDependencies()</code>	Loads the stored mode. Also re-runs when inherited dependencies change.
<code>build()</code>	Resolves <code>system</code> against real brightness and publishes the scope.

AppThemeScope — the pipe

A textbook `InheritedWidget` . Flutter keeps an index of these by type, so `dependOnInheritedWidgetOfExactType` is an $O(1)$ lookup no matter how deep the widget is — that's why this beats passing the theme through constructors.

```
lib/theme/app_theme_scope.dart
```

```
static AppThemeScope? of(BuildContext context) {
  final result = context.dependOnInheritedWidgetOfExactType<AppThemeScope>();
  assert(result != null, 'AppThemeScope not found in context');
  return result;
}
```

`dependOn...` does two things at once: it returns the widget, *and* it registers the calling widget as a dependent so it rebuilds when the scope changes.

06 – Why not one class?

The reason for the split

Each concern changes for a different reason, and at a different time.

IF YOU CHANGE...	YOU TOUCH...	EVERYTHING ELSE
Your brand colours	<code>AppColors</code> / <code>AppTheme</code>	Unaffected
Storage — Hive, secure storage, a server	<code>ThemeScopeWidget</code>	Unaffected
How the theme reaches widgets — Riverpod, Bloc	<code>AppThemeScope</code>	Unaffected
Adding a third mode, e.g. high contrast	<code>ThemeScopeWidget</code> + <code>AppTheme</code>	Scope unaffected

Fold all three into one class and every one of those changes touches the same file. That's the practical argument for the separation — not purity, but blast radius.

07 – Audit

Six things worth fixing

The architecture is sound. These are wiring problems inside it, ordered by how much they'd bite you.

Your light and dark `ThemeData` are identical DESIGN

In `app_template.dart:40–47`, `theme:` and `darkTheme:` are built from the same already-resolved `theme.theme` — the same four extensions, character for character. So `themeMode: on` `MaterialApp` has no effect on your colours. Dark mode still works, because `ThemeScopeWidget` already decided it upstream — but Material's own light/dark switch is inert configuration that reads as though it's doing something.

A fresh `AppTheme` is built on every rebuild PERFORMANCE

`AppTheme.light()` is called inside `build()`, so every rebuild allocates a new `AppColors`, a new `AppInputTheme`, a new `AppButtonTheme`, and **22 new `TextStyle` objects**. Since the result depends only on the mode and the brightness, cache the two bundles as statics and pick between them.

Two delivery paths for the same data REDUNDANCY

The theme reaches widgets through `AppThemeScope` and through `ThemeData.extensions`. Your `context.colors` getters use the second; the scope is read in only two places. Once the extensions are registered, `MaterialApp` distributes everything on its own — so the scope is close to redundant, and having both invites them to disagree.

`updateShouldNotify` always returns true PERFORMANCE

Every rebuild of the scope notifies every dependent, whether or not the theme actually changed. Comparing against `oldWidget` would fix it — though it only helps once `AppTheme` has real value equality, which today it doesn't, since a new instance is built each time. The two findings are worth fixing together.

Failures are swallowed silently DEBUGGING

on `Exception catch (_) {}` in `changeTo` has an empty body. If the write to preferences ever fails, the theme silently doesn't change and nothing is logged — the hardest kind of bug to chase. Log it, even if you choose not to surface it to the user.

`of()` returns nullable but asserts non-null ERGONOMICS

The assert already declares that null is a programming error, yet the return type stays nullable — so every call site carries a `!`. Returning `AppThemeScope` and throwing on absence would push the check into one place instead of all of them.

SAVE AS PDF

Print this page from your browser (⌘P) and choose *Save as PDF* — print styles are included, so it lays out with page breaks in sensible places and a light palette regardless of the theme you're reading in.